

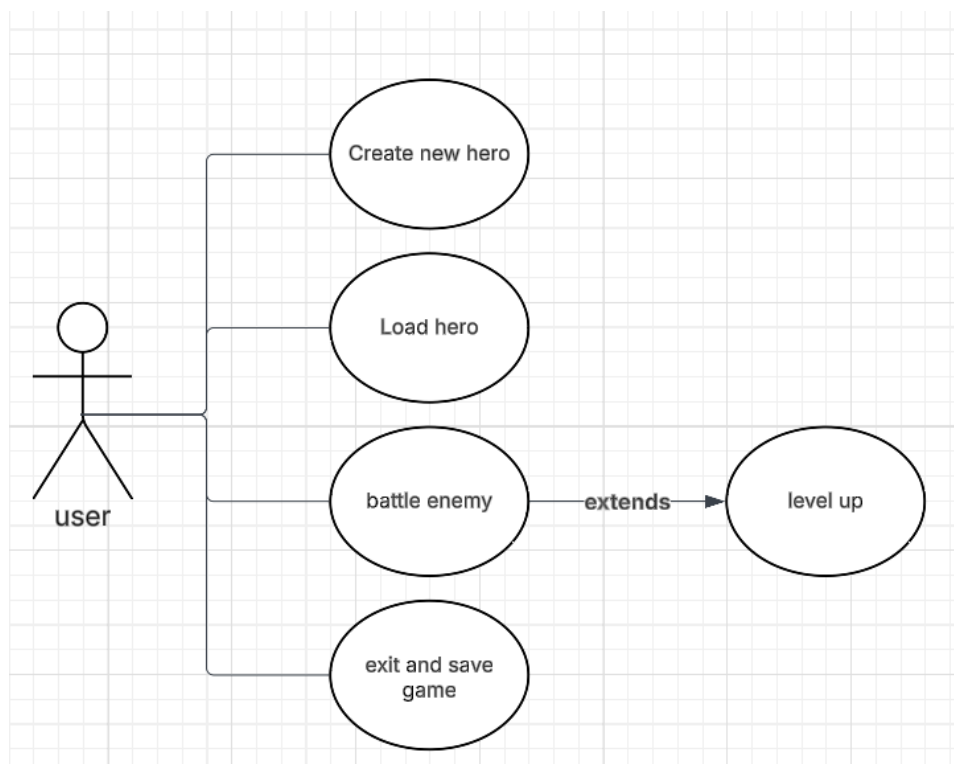
Iteration 1

Current program

The current program fulfills the first iteration requirements of the task. It makes a class for hero and enemy and a function for battling. Inside the hero class there are methods to show the hero's stats, to save the character in a file using the "fstream" library and to try to level up the character. The enemy class has only instance variables. The Battle function takes a hero and an enemy, the hero is actually a copy of the object since it's not referential. This is done to resets the hero's health after every battle. The function has a boolean return to show whether the player won or lost the battle.

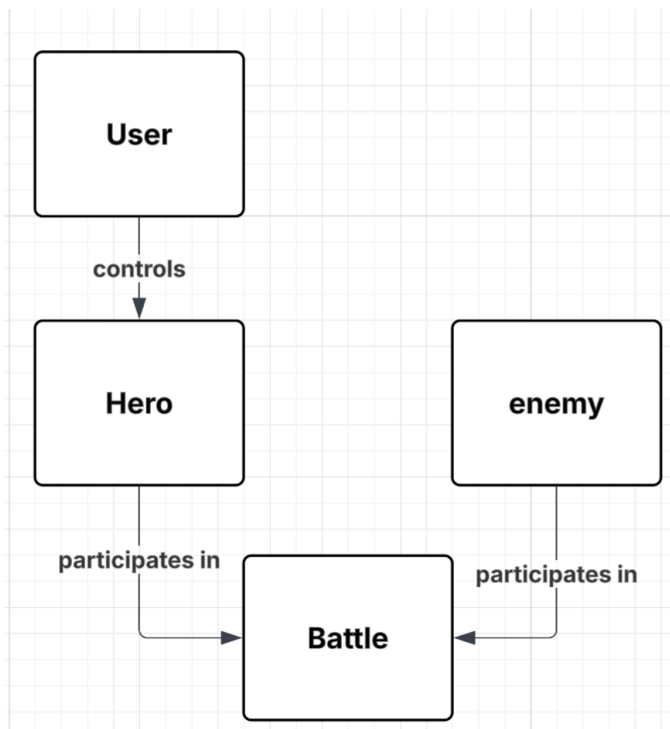
The game is played through the terminal with user inputs. The program asks the user whether they want to create a new character or load a previous one and the program enters a specific if-statement based on the user's response that does what the user wanted. The program then enters the main while-loop of the game where the user can choose to save and exit the game using the formerly mentioned methods or battle enemies using the battle function. If the user battles an enemy and wins, they get the xp reward from that enemy and the game asks again whether to exit the game or keep playing because of the while-loop.

Use case diagram



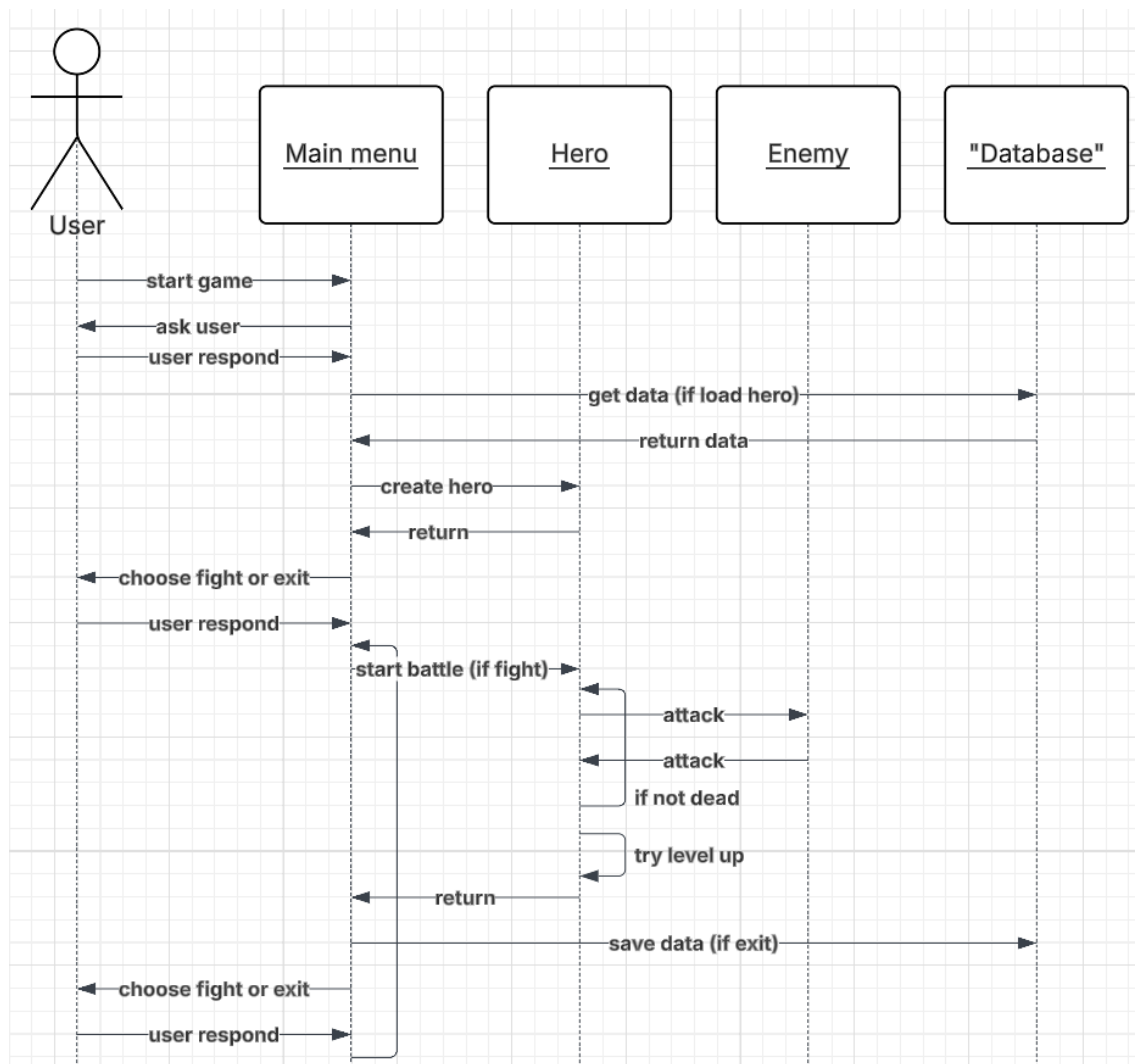
The player is the primary (and only) actor of the game. The "battle enemy" use case extends "level up" since that is a possibility (and not necessity) of battling.

Domain model



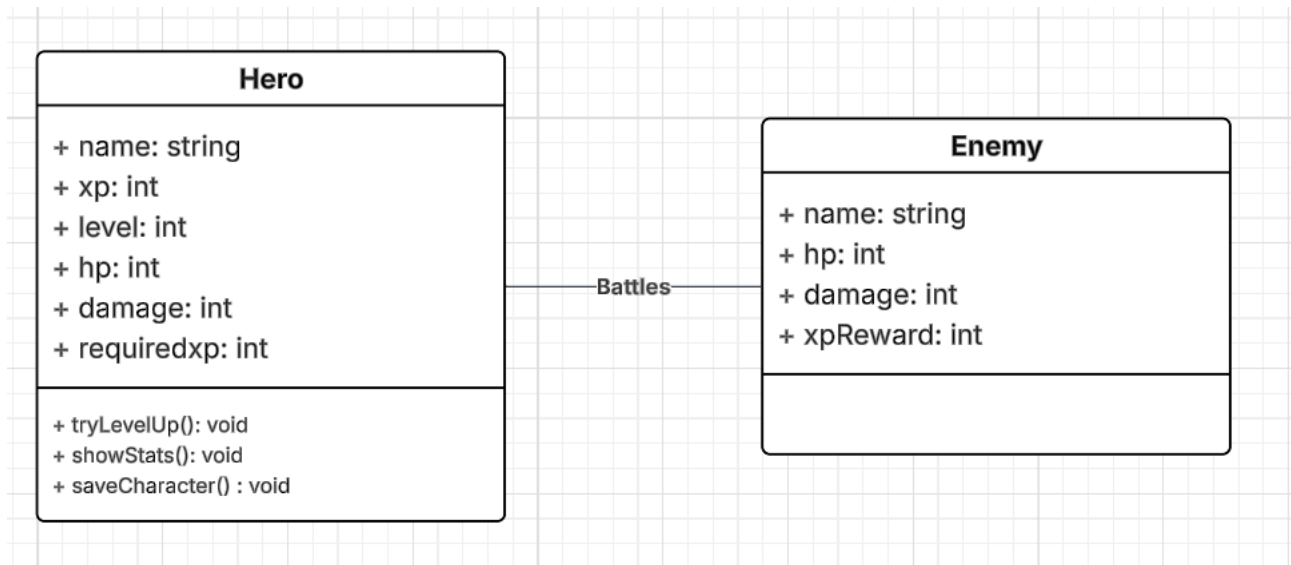
This domain model visualizes how the different parts of the program play together. The user controls the hero by controlling what enemy it fights and whether it fights at all. Both the hero and enemy participate in the battle.

Sequence diagram



Current sequence diagram with user input and the interactions between different parts of the program. Current “database” is composed of .txt files containing the heroes and their stats.

UML class diagram



This class diagram shows the different classes in the program and how they interact with each other. Currently there are only two classes, hero and enemy and they have an association with each other through the battle function but neither rely on the other. Since hero and enemy contain some of the same variables, I could have made an abstract superclass called “character” which hero and enemy would inherit from. This would have made the class diagram more interesting.

Iteration 2

Current program

Version 2 of the program includes new functions and classes. To make it easier to read and navigate the code, some functions and classes have been moved to different source and header files and included in the top of the main program. A new cave class has been added along with a function that makes a different cave based on user input. The cave class has a name instance variable which is being autogenerated by a method in the class using an array of nouns, an array of verbs and a randomizer function from the library “cstdlib”. The class also contains a vector of the enemy class and a method to add a certain enemy to the vector and how many times it should add that enemy to the vector.

The function to create the cave can create 4 different caves of varying difficulty and theme based on user input. The different caves contain different enemies and gold rewards. All enemy’s stats and the amount it creates of them also vary depending on the hero’s level.

Example:

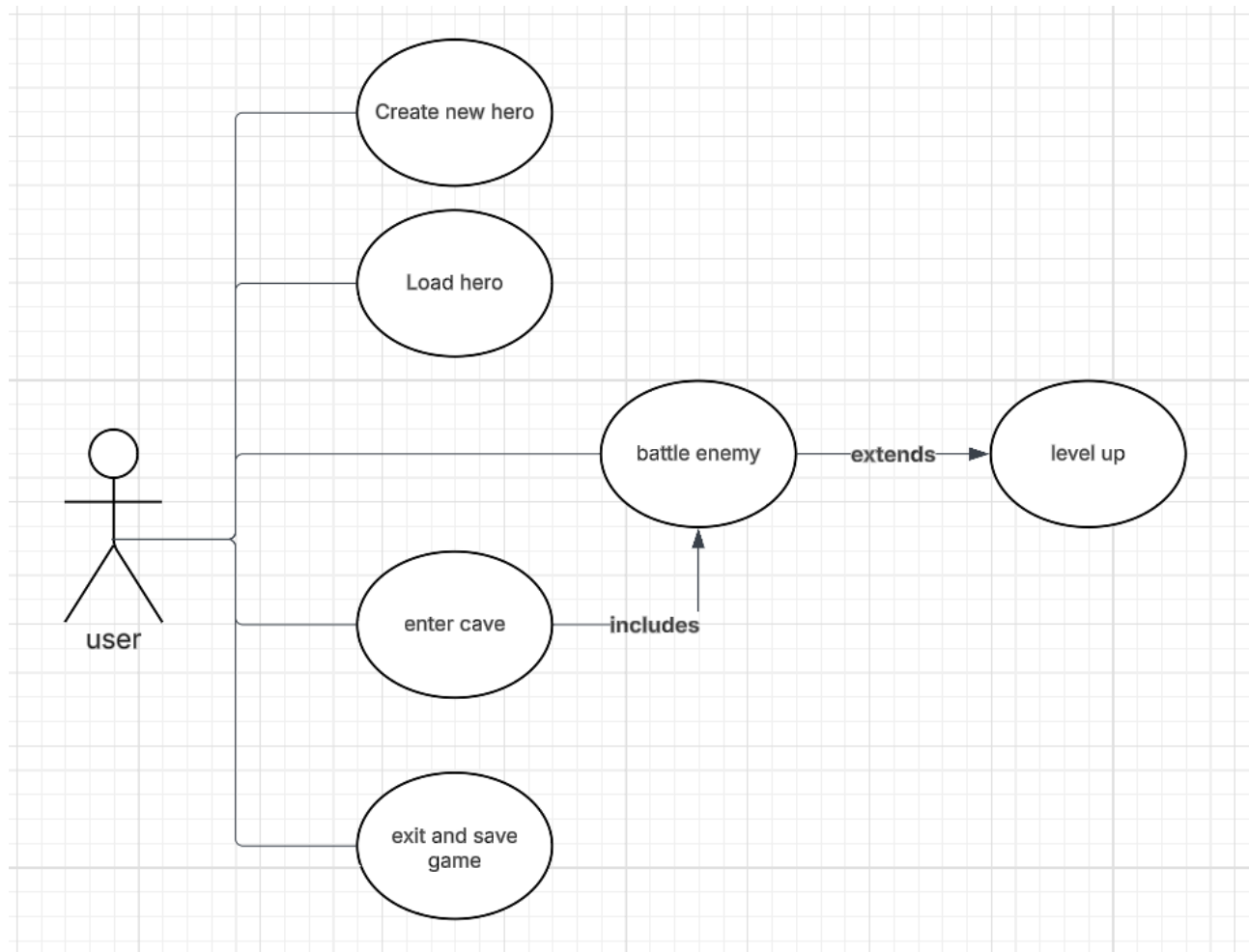
```
enemy bigskeleton({"Bone Guardian",50+10*(hero.level/2), 10+5*(hero.level/4), 4000});
cave.addEnemy(bigskeleton,(hero.level/10));
```

This example shows the function creating an enemy of varying difficulty based on hero level and adding that enemy to the cave. The addEnemy method’s second input is the number of times that

enemy should be added as an int. When you divide an int, it rounds down to the closest number meaning in this example, this enemy only unlocks for the hero at level 10.

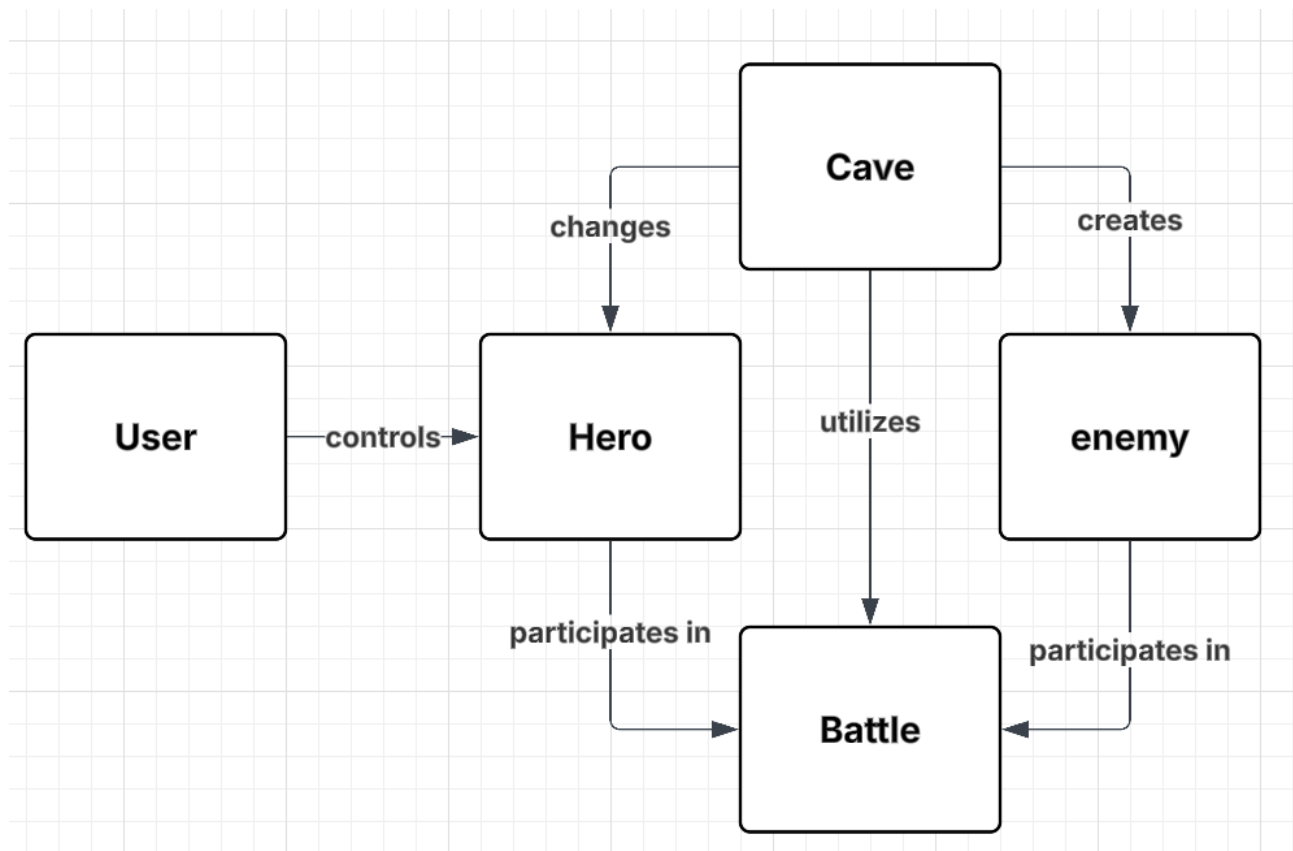
The user can still choose to fight enemies individually like in the first iteration.

Use case diagram



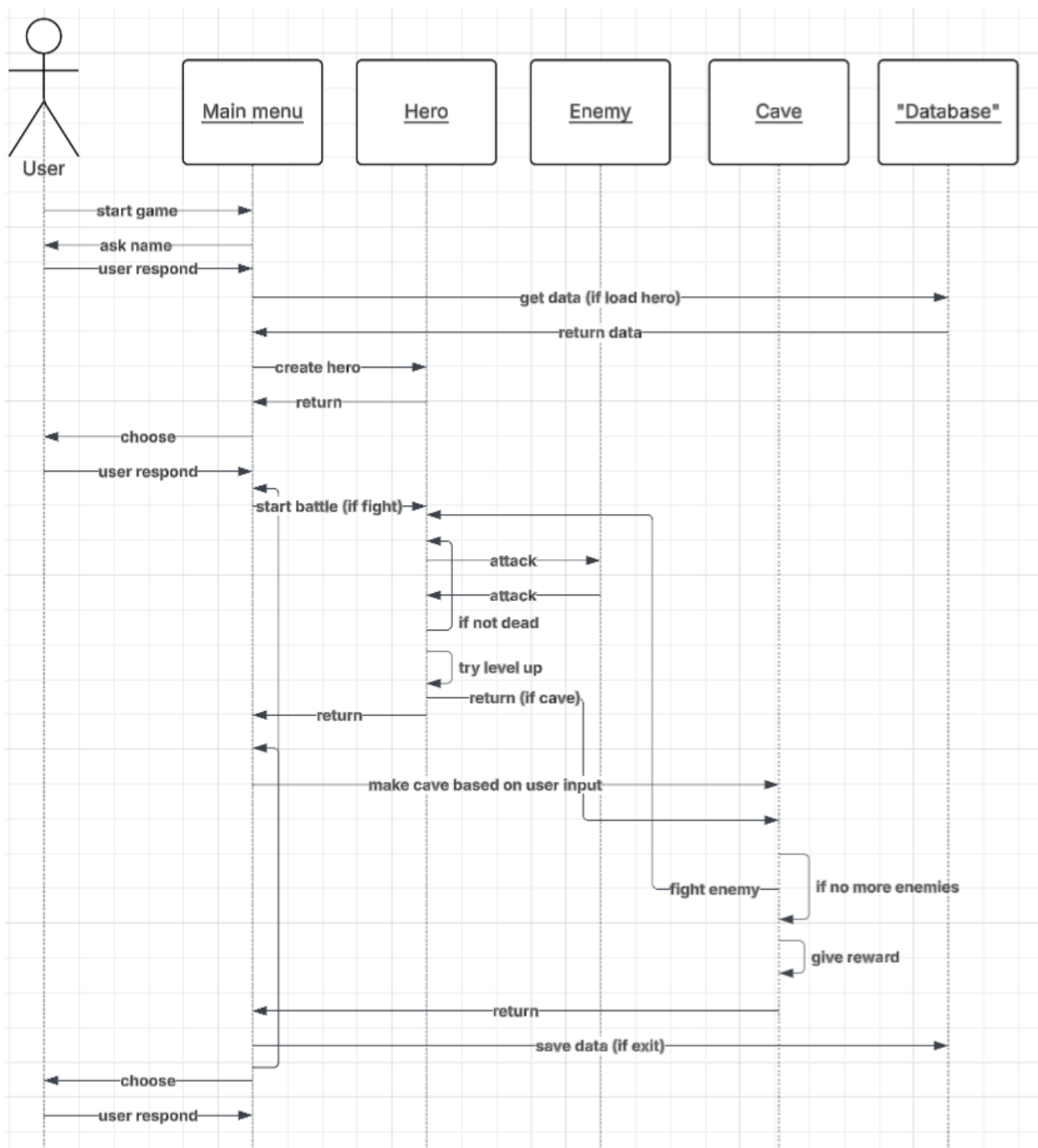
The new use case diagram features the use case “enter cave” which includes “battle enemy”.

Domain model

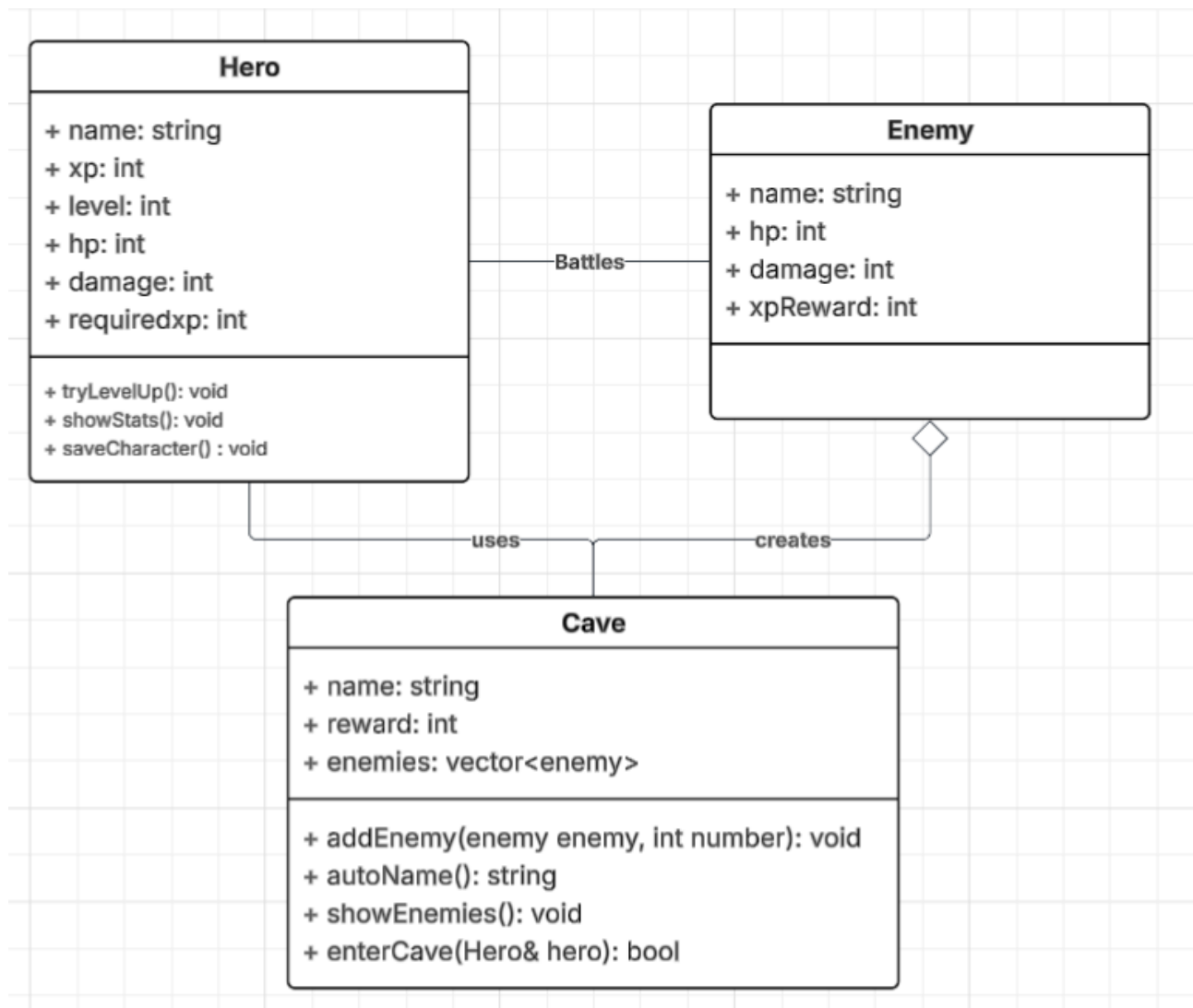


The new model adds the cave which uses the other parts in various ways. Cave creates several enemies, makes them fight the hero using the battle function and changes the hero in different ways. Note that user can still choose to fight enemies individually.

Sequence diagram



UML class diagram



The new case class aggregates the enemy class since it creates that class but both classes can exist without each other. Cave also has an association with hero since it uses that class in a method.

Iteration 3

Current system

In iteration 3, the program includes a new weapon class and several functions for managing the database and analyzing data from that database. The weapon class has variables for durability, strength, damage, price and kills. The class has a boolean method to subtract 1 from the durability when used and returning true if broken aka when durability reaches 0.

Changes to the hero class include three vectors of weapons from the weapon class and another weapon representing the current weapon the hero is using. The weapon vectors represent the hero's current weapons, unlocked weapons and locked weapons. Now when clearing a cave, a specific

weapon is part of the reward. The unlocked weapons vector gets a weapon from the locked weapons vector based on the cave cleared. The hero's damage is now calculated by a method when battling that includes the weapons damage and damage modifier.

The player can now choose to enter their armory to see their weapons, equip another weapon or buy a new weapon using gold earned from clearing caves.

A SQL database has been created from a SQL script included as a .txt file. The database stores data from the hero and weapon class.

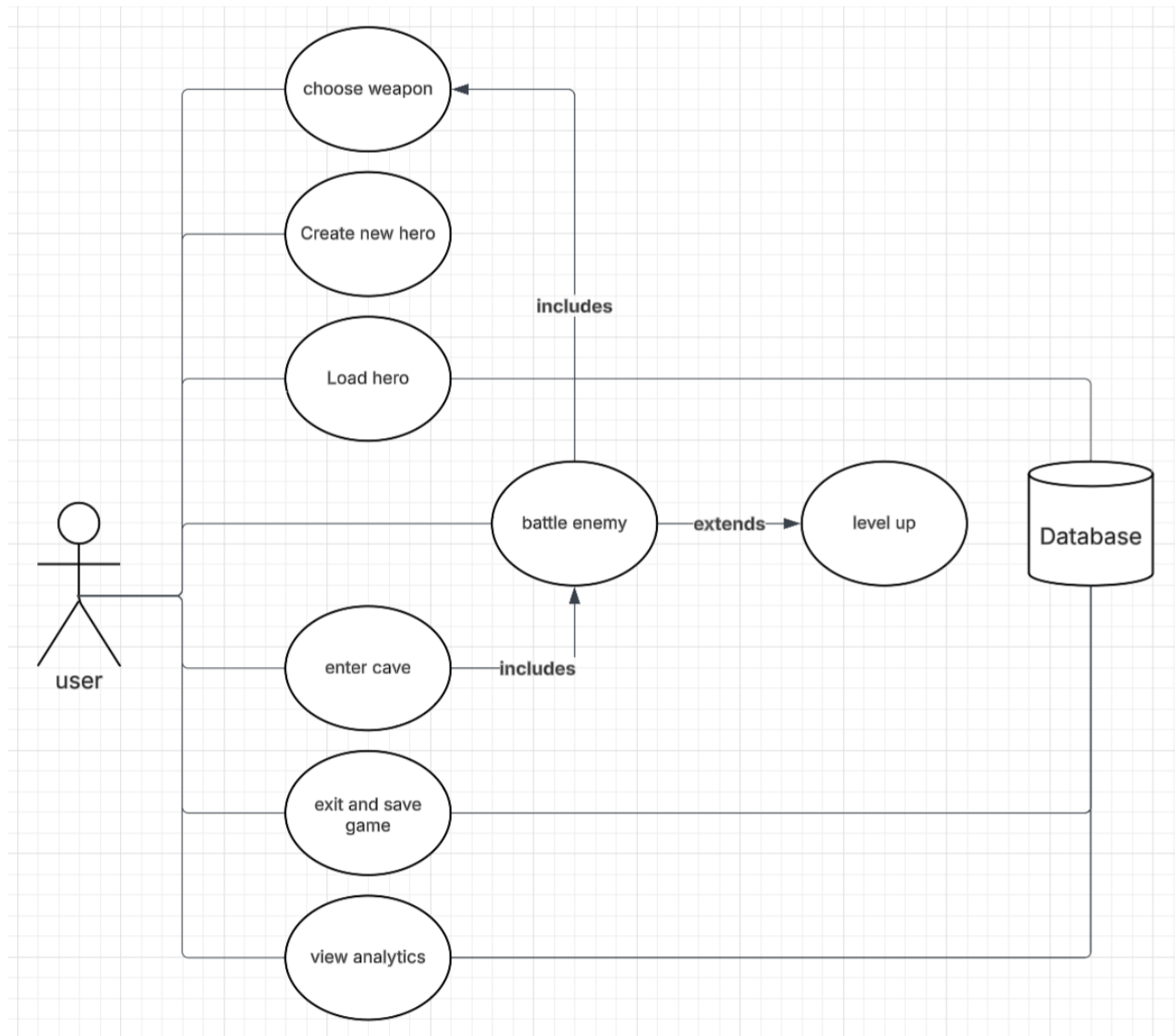
 Heroes	CREATE TABLE Heroes (id INTEGER PRIMARY KEY AUTOINCR	
 id	INTEGER	"id" INTEGER
 name	TEXT	"name" TEXT NOT NULL
 xp	INTEGER	"xp" INTEGER NOT NULL
 level	INTEGER	"level" INTEGER NOT NULL
 hp	INTEGER	"hp" INTEGER NOT NULL
 damage	INTEGER	"damage" INTEGER NOT NULL
 requiredxp	INTEGER	"requiredxp" INTEGER NOT NULL
 gold	INTEGER	"gold" INTEGER NOT NULL
 kills	INTEGER	"kills" INTEGER NOT NULL
 unlockedWeapons	INTEGER	"unlockedWeapons" INTEGER NOT NULL
 sqlite_sequence	CREATE TABLE sqlite_sequence(name,seq)	
 weapons	CREATE TABLE weapons (id INTEGER PRIMARY KEY AUTOINCR	
 id	INTEGER	"id" INTEGER
 heroID	INTEGER	"heroID" INTEGER NOT NULL
 name	TEXT	"name" TEXT NOT NULL
 damage	INTEGER	"damage" INTEGER NOT NULL
 strength	INTEGER	"strength" INTEGER NOT NULL
 durability	INTEGER	"durability" INTEGER NOT NULL
 kills	INTEGER	"kills" INTEGER NOT NULL
 price	INTEGER	"price" INTEGER NOT NULL

Both tables include an autoincrementing id as their primary keys instead of using their name. This allows for multiple heroes and weapons with the same name and makes it easier to load data in the program.

Functions have been added to save heroes and weapons in the database and to retrieve this data. These functions rely on the built-in QT-creator libraries related to SQL databases. Since you can't save a class as a datatype in the database, creative ways have been included in the functions to circumvent this and to be able to successfully load the data.

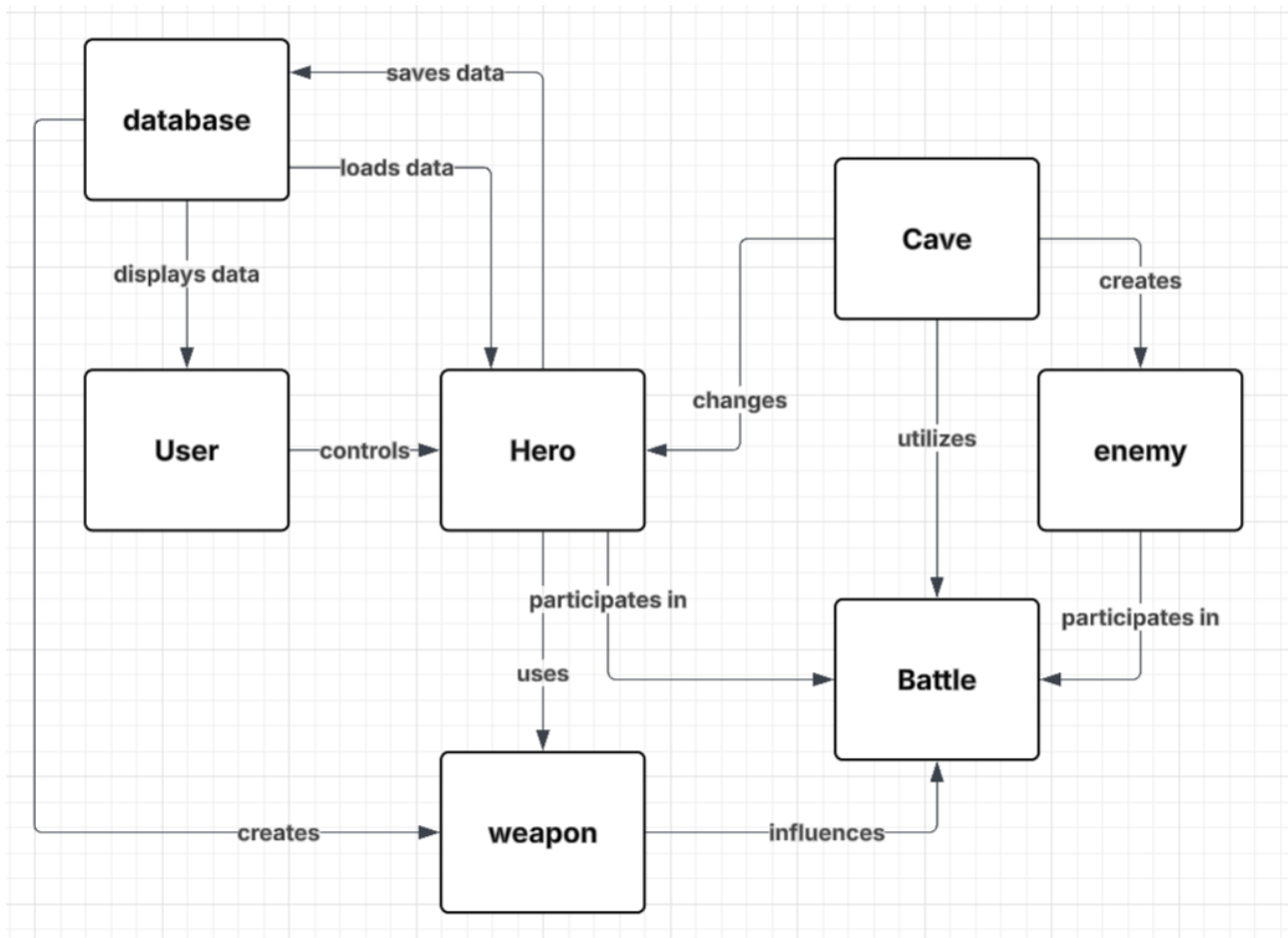
In the main menu, you can now choose to see the analytics based on previous characters from data in the database. These functions are similar to the function to load a character in that they use the libraries to access the database and use SQL to retrieve the data and sort it. There are functions to print all heroes alphabetically, how many monsters each hero has killed, a specific hero's kill count per weapon and every weapon and what hero has killed the most monsters with this weapon.

Use case diagram



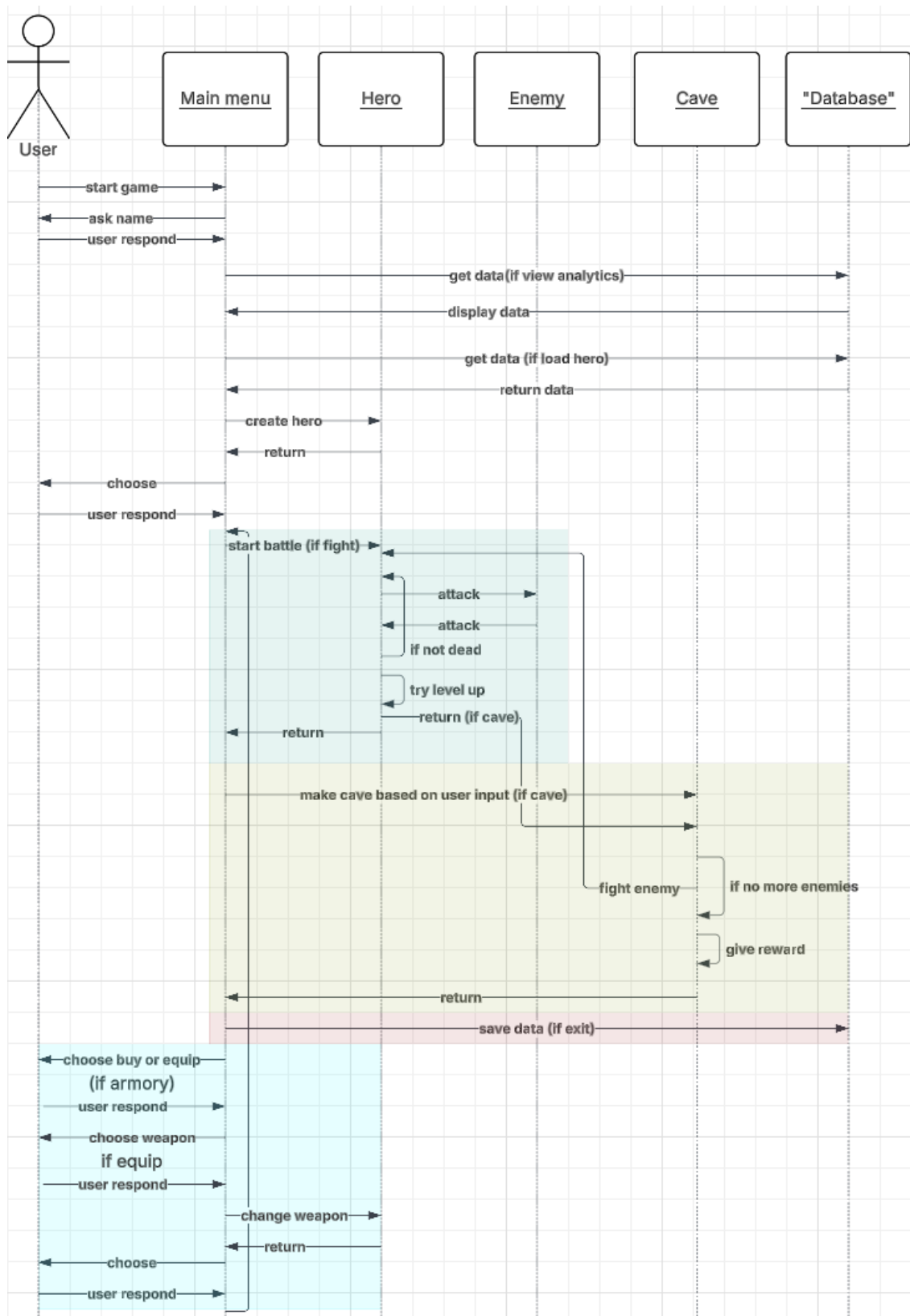
The updated use case diagram includes the database as another actor participating in the save and load game use cases and the view analytics use case.

Domain model



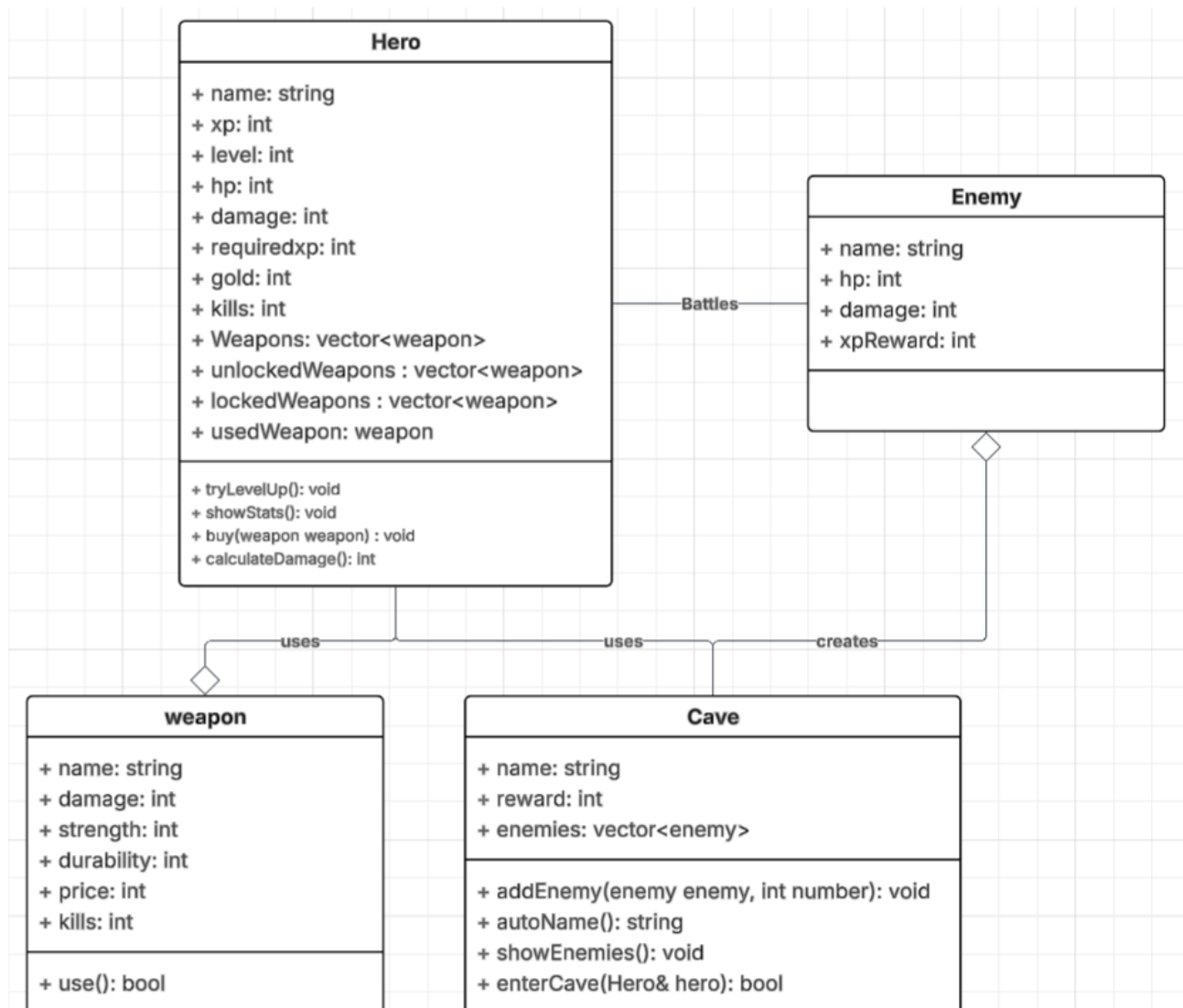
The new domain model includes the database and new weapon class. The database saves data from hero and loads data into hero. It also displays data analytics for the user and creates weapons for the hero. Hero uses weapons and weapons influence battles by effecting the hero's damage.

Sequence diagram



The updated sequence diagram contains the option to equip and buy new weapons for the hero. The options the user can enter have been colored to be easier distinguishable.

UML class diagram



The class diagram now contains the weapon class which aggregates from Hero as heroes can contain weapons but neither relies on each other for existing.